



18

CRASH RECOVERY

- What steps are taken in the ARIES method to recover from a DBMS crash?
- How is the log maintained during normal operation?
- How is the log used to recover from a crash?
- What information in addition to the log is used during recovery?
- What is a checkpoint and why is it used?
- What happens if repeated crashes occur during recovery?
- How is media failure handled?
- How does the recovery algorithm interact with concurrency control?
- Key concepts: steps in recovery, analysis, redo, undo; ARIES, repeating history; log, LSN, forcing pages, WAL; types of log records, update, commit, abort, end, compensation; transaction table, lastLSN; dirty page table, recLSN; checkpoint, fuzzy checkpointing, master log record; media recovery; interaction with concurrency control; shadow paging

Hurnpty Durnpty sat on a wall.
Hurnpty Durnpty had a great fall.
All the King's horses and all the King's men
Could not put Hurnpty together again.

—Old nursery rhyme

The recovery manager of a DBMS is responsible for ensuring two important properties of transactions: Atomicity and durability. It ensures *atomicity* by undoing the actions of transactions that do not commit and *durability* by making sure that all actions of committed transactions survive system crashes (e.g., a core dump caused by a bus error) and media failures (e.g., a disk is corrupted).

The recovery manager is one of the hardest components of a DBMS to design and implement. It must deal with a wide variety of database states because it is called on during system failures. In this chapter, we present the ARIES recovery algorithm, which is conceptually simple, works well with a wide range of concurrency control mechanisms, and is being used in an increasing number of database systems.

We begin with an introduction to ARIES in Section 18.1. We discuss the log, which is a central data structure in recovery, in Section 18.2, and other recovery-related data structures in Section 18.3. We complete our coverage of recovery-related activity during normal processing by presenting the Write-Ahead Logging protocol in Section 18.4, and checkpointing in Section 18.5.

We discuss recovery from a crash in Section 18.6. Aborting (or rolling back) a single transaction is a special case of Undo, discussed in Section 18.6.3. We discuss media failures in Section 18.7, and conclude in Section 18.8 with a discussion of the interaction of concurrency control and recovery and other approaches to recovery. In this chapter, we consider recovery only in a centralized DBMS; recovery in a distributed DBMS is discussed in Chapter 22.

18.1 INTRODUCTION TO ARIES

ARIES is a recovery algorithm designed to work with a steal, no-force approach. When the recovery manager is invoked after a crash, restart proceeds in three phases:

1. **Analysis:** Identifies dirty pages in the buffer pool (i.e., changes that have not been written to disk) and active transactions at the time of the crash.
2. **Redo:** Repeats all actions, starting from an appropriate point in the log, and restores the database state to what it was at the time of the crash.
3. **Undo:** Undoes the actions of transactions that did not commit, so that the database reflects only the actions of committed transactions.

Consider the simple execution history illustrated in Figure 18.1. When the system is restarted, the Analysis phase identifies *T1* and *T3* as transactions

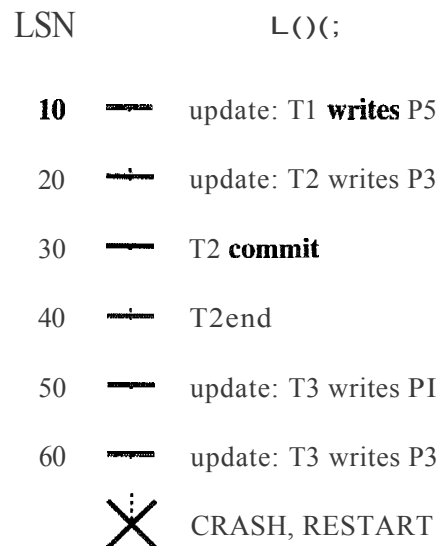


Figure 18.1 Execution History with a Crash

active at the time of the crash and therefore to be undone; *T2* as a committed transaction, and all its actions therefore to be written to disk; and *P1*, *P3*, and *P5* as potentially dirty pages. All the updates (including those of *T1* and *T3*) are reapplied in the order shown during the Redo phase. Finally, the actions of *T1* and *T3* are undone in reverse order during the Undo phase; that is, *T3*'s write of *P3* is undone, *T3*'s write of *P1* is undone, and then *T1*'s write of *P5* is undone.

Three main principles lie behind the ARIES recovery algorithm:

- **Write-Ahead Logging:** Any change to a database object is first recorded in the log; the record in the log must be written to stable storage before the change to the database object is written to disk.
- **Repeating History During Redo:** On restart following a crash, ARIES retraces all actions of the DBMS before the crash and brings the system back to the exact state that it was in at the time of the crash. Then, it undoes the actions of transactions still active at the time of the crash (effectively aborting them).
- **Logging Changes During Undo:** Changes made to the database while undoing a transaction are logged to ensure such an action is not repeated in the event of repeated (failures causing) restarts.

The second point distinguishes ARIES from other recovery algorithms and is the basis for much of its simplicity and flexibility. In particular, ARIES can support concurrency control protocols that involve locks of finer granularity than a page (e.g., record-level locks). The second and third points are also

Crash Recovery: IBM DB2, Informix, Microsoft SQL Server, Oracle 8, and Sybase ASE all use a WAL scheme for recovery. IBM DB2 uses ARIES, and the others use schemes that are actually quite similar to ARIES (e.g., all changes are re-applied, not just the changes made by transactions that are 'winners') although there are several variations.

important in dealing with operations where redoing and undoing the operation are not exact inverses of each other. We discuss the interaction between concurrency control and crash recovery in Section 18.8, where we also discuss other approaches to recovery briefly.

18.2 THELOG

The log, sometimes called the trail or journal, is a history of actions executed by the DBMS. Physically, the log is a file of records stored in stable storage, which is assumed to survive crashes; this durability can be achieved by maintaining two or more copies of the log on different disks (perhaps in different locations), so that the chance of all copies of the log being simultaneously lost is negligibly small.

The most recent portion of the log, called the log tail, is kept in main memory and is periodically forced to stable storage. This way, log records and data records are written to disk at the same granularity (pages or sets of pages).

Every log record is given a unique *id* called the log sequence number (LSN). As with any record id, we can fetch a log record with one disk access given the LSN. Further, LSNs should be assigned in monotonically increasing order; this property is required for the ARIES recovery algorithm. If the log is a sequential file, in principle growing indefinitely, the LSN can simply be the address of the first byte of the log record.¹

For recovery purposes, every page in the database contains the LSN of the most recent log record that describes a change to this page. This LSN is called the **pageLSN**.

A log record is written for each of the following actions:

¹In practice, various techniques are used to identify portions of the log that are 'too old' to be needed again to bound the amount of stable storage used for the log. Given such a bound, the log may be implemented as a 'circular' file, in which case the LSN may be the log record id plus a *wrap-count*.

- **Updating a Page:** After modifying the page, an *update* type record (described later in this section) is appended to the log tail. The pageLSN of the page is then set to the LSN of the update log record. (The page must be pinned in the buffer pool while these actions are carried out.)
- **Commit:** When a transaction decides to commit, it force-writes a *commit* type log record containing the transaction id. That is, the log record is appended to the log, and the log tail is written to stable storage, up to and including the commit record.² The transaction is considered to have committed at the instant that its commit log record is written to stable storage. (Some additional steps must be taken, e.g., reinserting the transaction's entry in the transaction table; these follow the writing of the commit log record.)
- **Abort:** When a transaction is aborted, an *abort* type log record containing the transaction id is appended to the log, and Undo is initiated for this transaction (Section 18.6.3).
- **End:** As noted above, when a transaction is aborted or committed, some additional actions must be taken beyond writing the abort or commit log record. After all these additional steps are completed, an *end* type log record containing the transaction id is appended to the log.
- **Undoing an update:** When a transaction is rolled back (because the transaction is aborted, or during recovery from a crash), its updates are undone. When the action described by an update log record is undone, a *compensation log record*, or CLR, is written.

Every log record has certain fields: prevLSN, transID, and type. The set of all log records for a given transaction is maintained as a linked list going back in time, using the prevLSN field; this list must be updated whenever a log record is added. The transID field is the id of the transaction generating the log record, and the type field obviously indicates the type of the log record.

Additional fields depend on the type of the log record. We already mentioned the additional contents of the various log record types, with the exception of the update and compensation log record types, which we describe next.

Update Log Records

The fields in an update log record are illustrated in Figure 18.2. The pageID field is the page id of the modified page; the length in bytes and the offset of the

²Note that this step requires the buffer manager to be able to selectively force pages to stable storage.

prevLSN	transID	type	pageID	length	offset	before-image	after-image
Fields common to all log records				Additional fields for update log records			

Figure 18.2 Contents of an Update Log Record

change are also included. The before-image is the value of the changed bytes before the change; the after-image is the value after the change. An update log record that contains both before- and after-images can be used to redo the change and undo it. In certain contexts, which we do not discuss further, we can recognize that the change will never be undone (or, perhaps, redone). A redo-only update log record contains just the after-image; similarly an undo-only update record contains just the before-image.

Compensation Log Records

A compensation log record (CLR) is written just before the change recorded in an update log record U is undone. (Such an undo can happen during normal system execution when a transaction is aborted or during recovery from a crash.) A compensation log record C describes the action taken to undo the actions recorded in the corresponding update log record and is appended to the log tail just like any other log record. The compensation log record C also contains a field called `undoNextLSN`, which is the LSN of the next log record that is to be undone for the transaction that wrote update record U ; this field in C is set to the value of `prevLSN` in U .

As an example, consider the fourth update log record shown in Figure 18.3. If this update is undone, a CLR would be written, and the information in it would include the `transID`, `pageID`, `length`, `offset`, and `before-image` fields from the update record. Notice that the CLR records the (undo) action of changing the affected bytes back to the before-image value; thus, this value and the location of the affected bytes constitute the redo information for the action described by the CLR. The `undoNextLSN` field is set to the LSN of the first log record in Figure 18.3.

Unlike an update log record, a CLR describes an action that will never be *undone*, that is, we never undo an undo action. The reason is simple: An update log record describes a change made by a transaction during normal execution and the transaction may subsequently be aborted, whereas a CLR describes an action taken to rollback a transaction for which the decision to abort has already been made. Therefore, the transaction *must* be rolled back, and the

undo action described by the CLR is definitely required. This observation is very useful because it bounds the amount of space needed for the log during restart from a crash: The number of CLRs that can be written during Undo is no more than the number of update log records for active transactions at the time of the crash.

A CLR may be written to stable storage (following WAL, of course) but the undo action it describes may not yet been written to disk when the system crashes again. In this case, the undo action described in the CLR is reapplied during the Redo phase, just like the action described in update log records.

For these reasons, a CLR contains the information needed to reapply, or redo, the change described but not to reverse it.

18.3 OTHER RECOVERY-RELATED STRUCTURES

In addition to the log, the following two tables contain important recovery-related information:

- **Transaction Table:** This table contains one entry for each active transaction. The entry contains (among other things) the transaction id, the status, and a field called **lastLSN**, which is the LSN of the most recent log record for this transaction. The status of a transaction can be that it is in progress, committed, or aborted. (In the latter two cases, the transaction will be removed from the table once certain 'clean up' steps are completed.)
- **Dirty page table:** This table contains one entry for each dirty page in the buffer pool, that is, each page with changes not yet reflected on disk. The entry contains a field **recLSN**, which is the LSN of the first log record that caused the page to become dirty. Note that this LSN identifies the earliest log record that might have to be redone for this page during restart from a crash.

During normal operation, these are maintained by the transaction manager and the buffer manager, respectively, and during restart after a crash, these tables are reconstructed in the Analysis phase of restart.

Consider the following simple example. Transaction T1000 changes the value of bytes 21 to 23 on page P500 from 'ABC' to 'DEF', transaction T2000 changes 'HIJ' to 'KLM' on page P600, transaction T2000 changes bytes 20 through 22 from 'GDE' to 'QRS' on page P500, then transaction T1000 changes 'TUV' to 'WXY' on page P505. The dirty page table, the transaction table,³ and

³The status field is not shown in the figure for space reasons; all transactions are in progress.

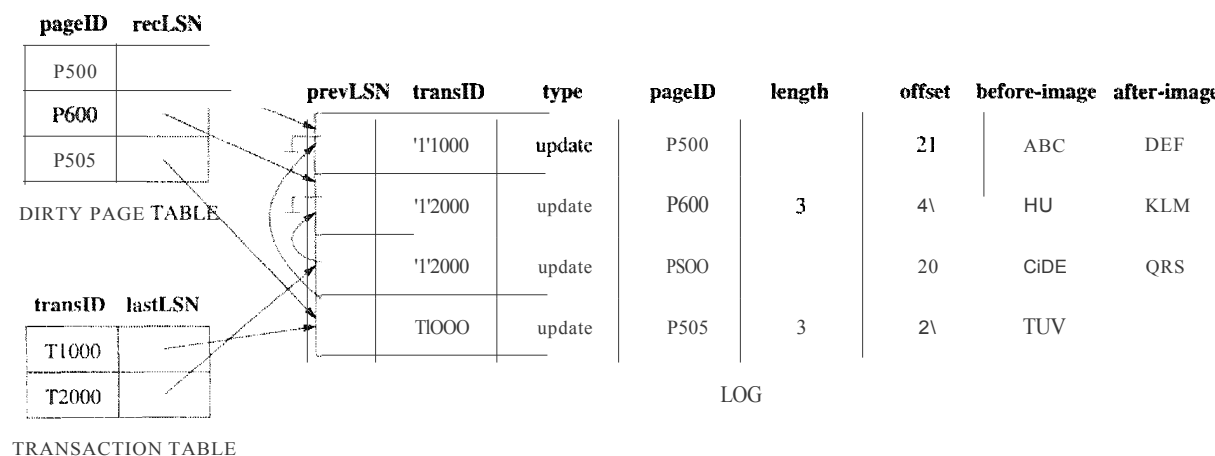


Figure 18.3 Instance of Log and Transaction Table

the log at this instant are shown in Figure 18.3. (Observe that the log is shown growing from top to bottom; older records are at the top. Although the records for each transaction are linked using the prevLSN field, the log as a whole also has a sequential order that is important—for example, T2000's change to page P500 follows T1000's change to page P500, and in the event of a crash, these changes must be redone in the same order.

18.4 THE WRITE-AHEAD LOG PROTOCOL

Before writing a page to disk, every update log record that describes a change to this page must be forced to stable storage. This is accomplished by forcing all log records up to and including the one with LSN equal to the pageLSN to stable storage before writing the page to disk.

The importance of the WAL protocol cannot be overemphasized—WAL is the fundamental rule that ensures that a record of every change to the database is available while attempting to recover from a crash. If a transaction made a change and committed, the no-force approach means that some of these changes may not have been written to disk at the time of a subsequent crash. Without a record of these changes, there would be no way to ensure that the changes of a committed transaction survive crashes. Note that the definition of a *committed transaction* is effectively 'a transaction all of whose log records, including a commit record, have been written to stable storage'.

When a transaction is committed, the log tail is forced to stable storage, even if a no-force approach is being used. It is worth contrasting this operation with the actions taken under a force approach: If a force approach is used, all the pages modified by the transaction, rather than a portion of the log that includes all its records, must be forced to disk when the transaction commits. The set of

all changed pages is typically much larger than the log tail because the size of an update log record is close to (twice) the size of the changed bytes, which is likely to be much smaller than the page size. Further, the log is maintained as a sequential file, and all writes to the log are sequential writes. Consequently, the cost of forcing the log tail is much smaller than the cost of writing all changed pages to disk.

18.5 CHECKPOINTING;

A checkpoint is like a snapshot of the DBMS state, and by taking checkpoints periodically, as we will see, the DBMS can reduce the amount of work to be done during restart in the event of a subsequent crash.

Checkpointing in ARIES has three steps. First, a `begin_checkpoint` record is written to indicate when the checkpoint starts. Second, an `end_checkpoint` record is constructed, including in it the current contents of the transaction table and the dirty page table, and appended to the log. The third step is carried out after the `end_checkpoint` record is written to stable storage: A special master record containing the LSN of the *begin_checkpoint* log record is written to a known place on stable storage. While the `end_checkpoint` record is being constructed, the DBMS continues executing transactions and writing other log records; the only guarantee we have is that the transaction table and dirty page table are accurate *as of the time of the begin_checkpoint record*.

This kind of checkpoint, called a fuzzy checkpoint, is inexpensive because it does not require quiescing the System or writing out pages in the buffer pool (unlike some other forms of checkpointing). On the other hand, the effectiveness of this checkpointing technique is limited by the earliest `recLSN` of pages in the dirty pages table, because during restart we must redo changes starting from the log record whose LSN is equal to this `recLSN`. Having a background process that periodically writes dirty pages to disk helps to limit this problem.

When the System comes back up after a crash, the restart process begins by locating the most recent checkpoint record. For uniformity, the system always begins normal execution by taking a checkpoint, in which the transaction table and dirty page table are both empty.

18.6 RECOVERING FROM A SYSTEM CRASH

When the system is restarted after a crash, the recovery manager proceeds in three phases, as shown in Figure 18.4.

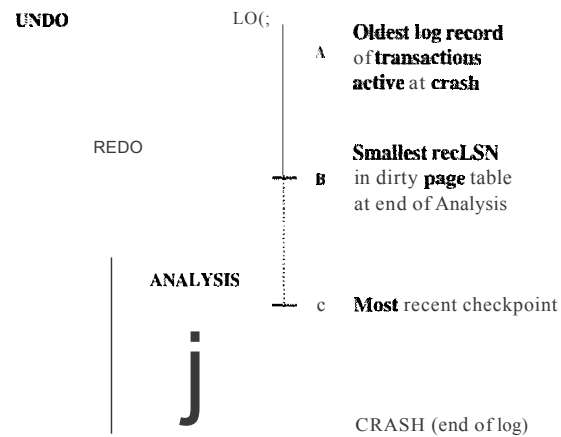


Figure 18.4 Three Phases of Restart in ARIES

The Analysis phase begins by examining the most recent begin_checkpoint record, whose LSN is denoted C in Figure 18.4, and proceeds forward in the log until the last log record. The Redo phase follows Analysis and redoes all changes to any page that might have been dirty at the time of the crash; this set of pages and the starting point for Redo (the smallest recLSN of any dirty page) are determined during Analysis. The Undo phase follows Redo and undoes the changes of all transactions active at the time of the crash; again, this set of transactions is identified during the Analysis phase. Note that Redo reapplies changes in the order in which they were originally carried out; Undo reverses changes in the opposite order, reversing the most recent change first.

Observe that the relative order of the three points A, B, and C in the log may differ from that shown in Figure 18.4. The three phases of restart are described in more detail in the following sections.

18.6.1 Analysis Phase

The Analysis phase performs three tasks:

1. It determines the point in the log at which to start the Redo pass.
2. It determines (a conservative superset of the) pages in the buffer pool that were dirty at the time of the crash.
3. It identifies transactions that were active at the time of the crash and must be undone.

Analysis begins by examining the most recent begin_checkpoint log record and initializing the dirty page table and transaction table to the copies of those structures in the next end_checkpoint record. Thus, these tables are initialized to the set of dirty pages and active transactions at the time of the checkpoint.

(If additional log records are between the `begin_checkpoint` and `end_checkpoint` records, the tables must be adjusted to reflect the information in these records, but we omit the details of this step. See Exercise 18.9.) Analysis then scans the log in the forward direction until it reaches the end of the log:

- If an end log record for a transaction T is encountered, T is removed from the transaction table because it is no longer active.
- If a log record other than an end record for a transaction T is encountered, an entry for T is added to the transaction table if it is not already there. Further, the entry for T is modified:
 1. The `lastLSN` field is set to the LSN of this log record.
 2. If the log record is a commit record, the status is set to C, otherwise it is set to U (indicating that it is to be undone).
- If a redoable log record affecting page P is encountered, and P is not in the dirty page table, an entry is inserted into this table with page id P and `recLSN` equal to the LSN of this redoable log record. This LSN identifies the oldest change affecting page P that may not have been written to disk.

At the end of the Analysis phase, the transaction table contains an accurate list of all transactions that were active at the time of the crash—this is the set of transactions with status U. The dirty page table includes all pages that were dirty at the time of the crash but may also contain some pages that were written to disk. If an *end_write* log record were written at the completion of each write operation, the dirty page table constructed during Analysis could be made more accurate, but in AHJES, the additional cost of writing *end_write* log records is not considered to be worth the gain.

As an example, consider the execution illustrated in Figure 18.3. Let us extend this execution by assuming that T_{2000} commits, then T_{1000} modifies another page, say, P_{700} , and appends an update record to the log tail, and then the system crashes (before this update log record is written to stable storage).

The dirty page table and the transaction table, held in memory, are lost in the crash. The most recent checkpoint was taken at the beginning of the execution, with an empty transaction table and dirty page table; it is not shown in Figure 18.3. After examining this log record, which we assume is just before the first log record shown in the figure, Analysis initializes the two tables to be empty. Scanning forward in the log, T_{1000} is added to the transaction table; in addition, P_{500} is added to the dirty page table with `recLSN` equal to the LSN of the first shown log record. Similarly, T_{2000} is added to the transaction table and P_{600} is added to the dirty page table. There is no change based on the third log record, and the fourth record results in the addition of P_{505} to

the dirty page table. The committed record for T2000 (not in the figure) is not encountered, and T2000 is removed from the transaction table.

The Analysis phase is now complete, and it is recognized that the only active transaction at the time of the crash is T1000, with lastLSN equal to the LSN of the fourth record in Figure 18.3. The dirty page table reconstructed in the Analysis phase is identical to that shown in the figure. The update log record for the change to P700 is lost in the crash and not seen during the Analysis pass. Thanks to the WAL protocol, however, all is well—the corresponding change to page P700 cannot have been written to disk either!

Some of the updates may have been written to disk; for concreteness, let us assume that the change to P600 (and only this update) was written to disk before the crash. Therefore P600 is not dirty, yet it is included in the dirty page table. The pageLSN on page P600, however, reflects the write because it is now equal to the LSN of the second update log record shown in Figure 18.3.

18.6.2 Redo Phase

During the Redo phase, ARIES reapplies the updates of *all* transactions, committed or otherwise. Further, if a transaction was aborted before the crash and its updates were undone, as indicated by CLRs, the actions described in the CLRs are also reapplied. This repeating history paradigm distinguishes ARIES from other proposed WAL-based recovery algorithms and causes the database to be brought to the same state it was in at the time of the crash.

The Redo phase begins with the log record that has the smallest recLSN of all pages in the dirty page table constructed by the Analysis pass because this log record identifies the oldest update that may not have been written to disk prior to the crash. Starting from this log record, Redo scans forward until the end of the log. For each redoable log record (update or CLR) encountered, Redo checks whether the logged action must be redone. The action must be redone unless one of the following conditions holds:

- The affected page is not in the dirty page table.
- The affected page is in the dirty page table, but the recLSN for the entry is *greater than* the LSN of the log record being checked.
- The pageLSN (stored on the page, which must be retrieved to check this condition) is *greater than or equal* to the LSN of the log record being checked.

The first condition obviously implies that all changes to this page have been written to disk. Because the recLSN is the first update to this page that may

not have been written to disk, the second condition means that the update being checked was indeed propagated to disk. The third condition, which is checked last because it requires us to retrieve the page, also ensures that the update being checked was written to disk, because either this update or a later update to the page was written. (Recall our assumption that a write to a page is atomic; this assumption is important here!)

If the logged action must be redone:

1. The logged action is reapplied.
2. The pageLSN on the page is set to the LSN of the redone log record. No additional log record is written at this time.

Let us continue with the example discussed in Section 18.6.1. From the dirty page table, the smallest recLSN is seen to be the LSN of the first log record shown in Figure 18.3. Clearly, the changes recorded by earlier log records (there happen to be none in this example) have been written to disk. Now, Redo fetches the affected page, *P500*, and compares the LSN of this log record with the pageLSN on the page and, because we assumed that this page was not written to disk before the crash, finds that the pageLSN is less. The update is therefore reapplied; bytes 21 through 23 are changed to 'DEF', and the pageLSN is set to the LSN of this update log record.

Redo then examines the second log record. Again, the affected page, *P600*, is fetched and the pageLSN is compared to the LSN of the update log record. In this case, because we assumed that *P600* was written to disk before the crash, they are equal, and the update does not have to be redone.

The remaining log records are processed similarly, bringing the system back to the exact state it was in at the time of the crash. Note that the first two conditions indicating that a redo is unnecessary never hold in this example. Intuitively, they come into play when the dirty page table contains a very old recLSN, going back to before the most recent checkpoint. In this case, as Redo scans forward from the log record with this LSN, it encounters log records for pages that were written to disk prior to the checkpoint and therefore not in the dirty page table in the checkpoint. Some of these pages may be dirtied again after the checkpoint; nonetheless, the updates to these pages prior to the checkpoint need not be redone. Although the third condition alone is sufficient to recognize that these updates need not be redone, it requires us to fetch the affected page. The first two conditions allow us to recognize this situation without fetching the page. (The reader is encouraged to construct examples that illustrate the use of each of these conditions; see Exercise 18.8.)

At the end of the Redo phase, end type records are written for all transactions with status C, which are rCllloved '£l'0ln the transaction table.

18.6.3 Undo Phase

The Undo phase, unlike the other two phases, scans backward from the end of the log. The goal of this phase is to undo the actions of all transactions active at the time of the crash, that is, to effectively abort them. This set of transactions is identified in the transaction table constructed by the Analysis phase.

The Undo Algorithm

Undo begins with the transaction table constructed by the Analysis phase, which identifies all transactions active at the time of the crash, and includes the LSN of the most recent log record (the lastLSN field) for each such transaction. Such transactions are called loser transactions. All actions of losers must be undone, and further, these actions must be undone in the reverse of the order in which they appear in the log.

Consider the set of lastLSN values for all loser transactions. Let us call this set ToUndo. Undo repeatedly chooses the largest (i.e., most recent) LSN value in this set and processes it, until ToUndo is empty. To process a log record:

1. If it is a CLR and the undoNextLSN value is not *null*, the undoNextLSN value is added to the set ToUndo; if the undoNextLSN is *null*, an end record is written for the transaction because it is completely undone, and the CLR is discarded.
2. If it is an update record, a CLR is written and the corresponding action is undone, as described in Section 18.2, and the prevLSN value in the update log record is added to the set ToUndo.

When the set ToUndo is empty, the Undo phase is complete. The system is now complete, and the system can proceed with normal operations.

Let us continue with the scenario discussed in Sections 18.6.1 and 18.6.2. The only active transaction at the time of the crash was determined to be T1000. From the transaction table, we get the LSN of its most recent log record, which is the fourth update log record in Figure 18.3. The update is undone, and a CLR is written with undoNextLSN equal to the LSN of the first log record in the figure. The next record to be undone for transaction T1000 is the first log record in the figure. After this is undone, a CLR and an end log record for T1000 are written, and the Undo phase is complete.

In this example, undoing the action recorded in the first log record causes the action of the third log record, which is due to a committed transaction, to be overwritten and thereby lost! This situation arises because T_{2000} overwrote a data item written by T_{1000} while T_{1000} was still active; if Strict 2PL were followed, T_{2000} would not have been allowed to overwrite this data item.

Aborting a Transaction

Aborting a transaction is just a special case of the Undo phase of Restart in which a single transaction, rather than a set of transactions, is undone. The example in Figure 18.5, discussed next, illustrates this point.

Crashes during Restart

It is important to understand how the Undo algorithm presented in Section 18.6.3 handles repeated system crashes. Because the details of precisely how the action described in an update log record is undone are straightforward, we discuss Undo in the presence of system crashes using an execution history, shown in Figure 18.5, that abstracts away unnecessary detail. This example illustrates how aborting a transaction is a special case of Undo and how the use of CLRs ensures that the Undo action for an update log record is not applied twice.

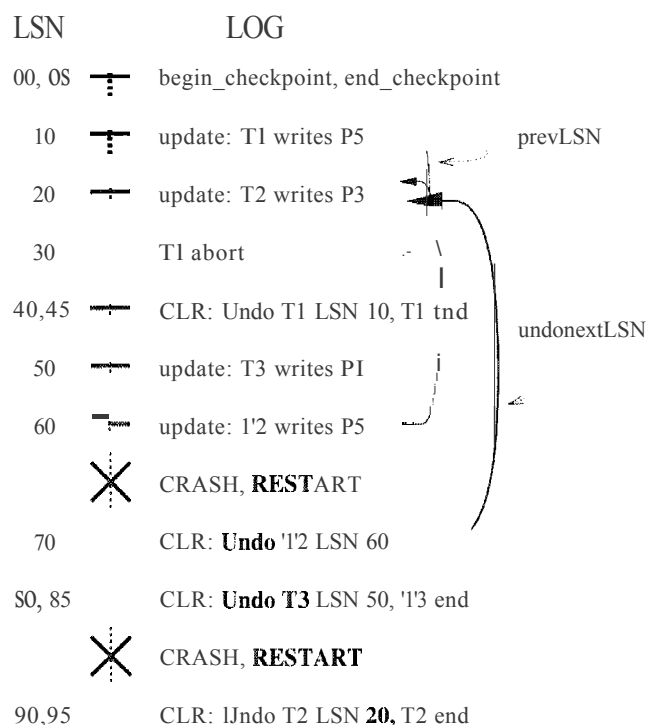


Figure 18.5 Example of Undo with Repeated Crashes

The log shows the order in which the DBMS executed various actions; note that the LSNs are in ascending order, and that each log record for a transaction has a prevLSN' field that points to the previous log record for that transaction. We have not shown *null* prevLSNs, that is, some special value used in the prevLSN field of the first log record for a transaction to indicate that there is no previous log record. We also compacted the figure by occasionally displaying two log records (separated by a column) on a single line.

Log record (with LSN) 30 indicates that *T1* aborts. All actions of this transaction should be undone in reverse order, and the only action of *T1*, described by the update log record 10, is indeed undone as indicated by CLR, 40.

After the first crash, Analysis identifies *P1* (with recLSN 50), *P3* (with recLSN 20), and *P5* (with recLSN 10) as dirty pages. Log record 45 shows that *T1* is a completed transaction; hence, the transaction table identifies *T2* (with lastLSN 60) and *T3* (with lastLSN 50) as active at the time of the crash. The Redo phase begins with log record 10, which is the minimum recLSN in the dirty page table, and reapplies all actions (for the update and CLR records), as per the Redo algorithm presented in Section 18.6.2.

The redo set consists of LSNs 60, for *T2*, and 50, for *T3*. The redo phase now begins by processing the log record with LSN 60 because 60 is the largest LSN in the ToUndo set. The update is undone, and a CLR, (with LSN 70) is written to the log. This CLR has undoNextLSN equal to 20, which is the prevLSN value in log record 60; 20 is the next action to be undone for *T2*. Now the largest remaining LSN in the ToUndo set is 50. The write corresponding to log record 50 is now undone, and a CLR, describing the change is written. This CLR has LSN 80, and its undoNextLSN field is *null* because 50 is the only log record for transaction *T3*. Therefore *T3* is completely undone, and an end record is written. Log records 70, 80, and 85 are written to stable storage before the system crashes a second time; however, the changes described by these records may not have been written, to disk.

When the system is restarted after the second crash, Analysis determines that the only active transaction at the time of the crash was *T2*; in addition, the dirty page table is identical to what it was during the previous restart. Log records 10 through 85 are processed again during Redo. (If some of the changes made during the previous Redo were written to disk, the pageLSN's on the affected pages are used to detect this situation and avoid writing these pages again.) The redo phase considers the only LSN in the ToUndo set, 70, and processes it by adding the undoNextLSN value (20) to the ToUndo set. Next, log record 20 is processed by undoing *T2*'s write of page *P3*, and a CLR is written (LSN 90). Because 20 is the first of *T2*'s log records and therefore, the last of its records

to be undone—the undoNextLSN field in this CLR is *null*, an end record is written for T_2 , and the ToUndo set is now empty.

Recovery is now complete, and normal execution can resume with the writing of a checkpoint record.

This example illustrated repeated crashes during the Undo phase. For completeness, let us consider what happens if the system crashes while Restart is in the Analysis or Redo phase. If a crash occurs during the Analysis phase, all the work done in this phase is lost, and on restart the Analysis phase starts afresh with the same information as before. If a crash occurs during the Redo phase, the only effect that survives the crash is that some of the changes made during Redo may have been written to disk prior to the crash. Restart starts again with the Analysis phase and then the Redo phase, and some update log records that were redone the first time around will not be redone a second time because the pageLSN is now equal to the update record's LSN (although the pages have to be fetched again to detect this).

We can take checkpoints during Restart to minimize repeated work in the event of a crash, but we do not discuss this point.

18.7 MEDIA RECOVERY

Media recovery is based on periodically making a copy of the database. Because copying a large database object such as a file can take a long time, and the DBMS must be allowed to continue with its operations in the meantime, creating a copy is handled in a manner similar to taking a fuzzy checkpoint.

When a database object such as a file or a page is corrupted, the copy of that object is brought up-to-date by using the log to identify and reapply the changes of committed transactions and undo the changes of uncommitted transactions (as of the time of the media recovery operation).

The begin_checkpoint LSN of the most recent complete checkpoint is recorded along with the copy of the database object to minimize the work in reapplying changes of committed transactions. Let us compare the smallest recLSN of a dirty page in the corresponding end_checkpoint record with the LSN of the begin_checkpoint record and call the smaller of these two LSNs I . We observe that the actions recorded in all log records with LSNs less than I must be reflected in the copy. Thus, only log records with LSNs greater than I need be reapplied to the copy.

Finally, the updates of transactions that are incomplete at the time of Inedia recovery or that were aborted after the fuzzy copy was completed need to be undone to ensure that the page reflects only the actions of committed transactions. The set of such transactions can be identified as in the Analysis pass, and we omit the details.

18.8 OTHER APPROACHES AND INTERACTION WITH CONCURRENCY CONTROL

Like ARIES, the most popular alternative recovery algorithms also maintain a log of database actions according to the WAL protocol. A major distinction between ARIES and these variants is that the Redo phase in ARIES *repeats history*, that is, redoes the actions of *all* transactions, not just the non-losers. Other algorithms redo only the non-losers, and the Redo phase follows the Undo phase, in which the actions of losers are rolled back.

Thanks to the repeating history paradigm and the use of CLRs, ARIES supports fine-granularity locks (record-level locks) and logging of logical operations rather than just byte-level modifications. For example, consider a transaction T that inserts a data entry 15* into a B+ tree index. Between the time this insert is done and the time that T is eventually aborted, other transactions may also insert and delete entries from the tree. If record-level locks are set rather than page-level locks, the entry 15* may be on a different physical page when T aborts from the one that T inserted it into. In this case, the undo operation for the insert of 15* must be recorded in logical terms because the physical (byte-level) actions involved in undoing this operation are not the inverse of the physical actions involved in inserting the entry.

Logging logical operations yields considerably higher concurrency, although the use of fine-granularity locks can lead to increased locking activity (because more locks must be set). Hence, there is a trade-off between different WAL-based recovery schemes. We chose to cover ARIES because it has several attractive properties, in particular, its simplicity and its ability to support fine-granularity locks and logging of logical operations.

One of the earliest recovery algorithms, used in the System R prototype at IBM¹, takes a very different approach. There is no logging and, of course, no WAL protocol. Instead, the database is treated as a collection of pages and accessed through a page table, which maps page ids to disk addresses. When a transaction makes changes to a data page, it actually makes a copy of the page, called the shadow of the page, and changes the shadow page. The transaction copies the appropriate part of the page table and changes the entry for the changed page to point to the shadow, so that it can see the

changes; however, other transactions continue to see the original page table, and therefore the original page, until this transaction commits. Aborting a transaction is simple: just discard its shadow versions of the page table and the data pages. Committing a transaction involves making its version of the page table public and discarding the original data pages that are superseded by shadow pages.

This scheme suffers from a number of problems. First, data becomes highly fragmented due to the replacement of pages by shadow versions, which may be located far from the original page. This phenomenon reduces data clustering and makes good garbage collection imperative. Second, the scheme does not yield a sufficiently high degree of concurrency. Third, there is a substantial storage overhead due to the use of shadow pages. Fourth, the process aborting a transaction can itself run into deadlocks, and this situation must be specially handled because the semantics of aborting an abort transaction gets murky.

For these reasons, even in System R, shadow paging was eventually superseded by WAL-based recovery techniques.

18.9 REVIEW QUESTIONS

Answers to the review questions can be found in the listed sections.

- What are the advantages of the ARIES recovery algorithm? (Section 18.1)
- Describe the three steps in crash recovery in ARIES? What is the goal of the Analysis phase? The redo phase? The undo phase? (Section 18.1)
- What is the LSN of a log record? (Section 18.2)
- What are the different types of log records and when are they written? (Section 18.2)
- What information is maintained in the transaction table and the dirty page table? (Section 18.3)
- What is Write-Ahead Logging? What is forced to disk at the time a transaction commits? (Section 18.4)
- What is a fuzzy checkpoint? Why is it useful? What is a master log record? (Section 18.5)
- In which direction does the Analysis phase of recovery scan the log? At which point in the log does it begin and end the scan? (Section 18.6.1)
- Describe what information is gathered in the Analysis phase and how. (Section 18.6.1)

- In which direction does the Redo phase of recovery process the log? At which point in the log does it begin and end? (Section 18.6.2)
- What is a redoable log record? Under what conditions is the logged action redone? What steps are carried out when a logged action is redone? (Section 18.6.2)
- In which direction does the Undo phase of recovery process the log? At which point in the log does it begin and end? (Section 18.6.3)
- What are loser transactions? How are they processed in the Undo phase and in what order? (Section 18.6.3)
- Explain what happens if there are crashes during the Undo phase of recovery. What is the role of CLRs? What if there are crashes during the Analysis and Redo phases? (Section 18.6.3)
- How does a DBMS recover from media failure without reading the complete log? (Section 18.7)
- Record-level logging increases concurrency. What are the potential problems, and how does ARIES address them? (Section 18.8)
- What is shadow paging? (Section 18.8)

EXERCISES

Exercise 18.1 Briefly answer the following questions:

1. How does the recovery manager ensure atomicity of transactions? How does it ensure durability?
2. What is the difference between stable storage and disk?
3. What is the difference between a system crash and a media failure?
4. Explain the WAL protocol.
5. Describe the steal and no-force policies.

Exercise 18.2 Briefly answer the following questions:

1. What are the properties required of LSNs?
2. What are the fields in an update log record? Explain the use of each field.
3. What are redoable log records?
4. What are the differences between update log records and CLRs?

Exercise 18.3 Briefly answer the following questions:

1. What are the roles of the Analysis, Redo, and Undo phases in ARIES?
2. Consider the execution shown in Figure 18.6.

Crash Recovery

LSN		LOG
00		begin_checkpoint
10	■	end_checkpoint
20	■	update: T1 writes P5
30	⊢	update: T2 writes P3
40	⊢	T2 commit
50		T2end
60	⊢	update: T3 writes P3
70		T1 abort
	⊗	CRASH, RESTART

Figure 18.6 Execution with a Crash

LSN		LOG
00	⊢	update: T1 writes P2
10	⊢	update: T1 writes P1
20	■	update: T2 writes P5
30		update: T3 writes P3
40		T3 commit
50	⊢	update: T2 writes P5
60	⊢	update: T2 writes P3
70	⊢	T2 abort

Figure 18.7 Aborting a Transaction

- What is done during Analysis? (Be precise about the points at which Analysis begins and ends and describe the contents of any tables constructed in this phase.)
- What is done during Redo? (Be precise about the points at which Redo begins and ends.)
- What is done during Undo? (Be precise about the points at which Undo begins and ends.)

Exercise 18.4 Consider the execution shown in Figure 18.7.

- Extend the figure to show prevLSN and lndonextLSN values.
- Describe the actions taken to rollback transaction T2.

LSN		LOG
00	—	begin_checkpoint
10		end_checkpoint
20	⊥	update: l'1 writes P1
30	⊥	update: l'2 writes P2
40	⊥	update: l'3 writes P3
50	—	l'2 commit
60	⊥	update: l'3 writes P2
70	⊥	l'2 end
80	—	update: l'1 writes P5
90	⊥	l'3 abort
	⊗	CRASH,RESTART

Figure 18.8 Execution with Multiple Crashes

3. Show the log after T_2 is rolled back, including all prevLSN and undonextLSN values in log records.

Exercise 18.5 Consider the execution shown in Figure 18.8. In addition, the system crashes during recovery after writing two log records to stable storage and again after writing another two log records.

1. What is the value of the LSN stored in the master log record?
2. What is done during Analysis?
3. What is done during Redo?
4. What is done during Undo?
5. Show the log when recovery is complete, including all non-null prevLSN and undonextLSN values in log records.

Exercise 18.6 Briefly answer the following questions:

1. How is checkpointing done in ARIES?
2. Checkpointing can also be done as follows: Quiesce the system so that only checkpointing activity can be in progress, write out copies of all dirty pages, and include the dirty page table and transaction table in the checkpoint record. What are the pros and cons of this approach versus the checkpointing approach of ARIES?
3. What happens if a second begin_checkpoint record is encountered during the Analysis phase?
4. Can a second end_checkpoint record be encountered during the Analysis phase?
5. Why is the use of CLRs important for the use of undo actions that are not the physical inverse of the original update?

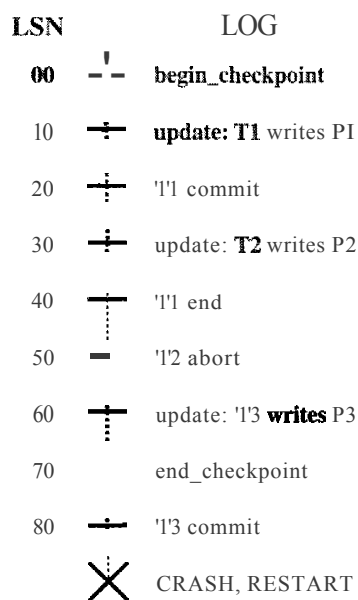


Figure 18.9 Log Records between Checkpoint Records

6. Give an example that illustrates how the paradigm of repeating history and the use of CLRs allow ARIES to support locks of finer granularity than a page.

Exercise 18.7 Briefly answer the following questions:

1. If the system fails repeatedly during recovery, what is the maximum number of log records that can be written (as a function of the number of update and other log records written before the crash) before restart completes successfully?
2. What is the oldest log record we need to retain?
3. If a bounded amount of stable storage is used for the log, how can we always ensure enough stable storage to hold all log records written during restart?

Exercise 18.8 Consider the three conditions under which a redo is unnecessary (Section 20.2.2).

1. Why is it cheaper to test the first two conditions?
2. Describe an execution that illustrates the use of the first condition.
3. Describe an execution that illustrates the use of the second condition.

Exercise 18.9 The description in Section 18.6.1 of the Analysis phase made the simplifying assumption that no log records appeared between the begin_checkpoint and end_checkpoint records for the most recent complete checkpoint. The following questions explore how such records should be handled.

1. Explain why log records could be written between the begin_checkpoint and end_checkpoint records.
2. Describe how the Analysis phase could be modified to handle such records.
3. Consider the execution shown in Figure 18.9. Show the contents of the end_checkpoint record.
4. Illustrate your modified Analysis phase on the execution shown in Figure 18.9.

Exercise 18.10 Answer the following questions briefly:

1. Explain how Inedin recovery is handled in ARIES.
2. What are the pros and cons of using fuzzy durnps for media recovery?
3. What are the sirYlilarities and differences between checkpoints and fuzzy chunps?
4. Contrast ARIES with other WAL-based recovery schernes.
5. Contrast AHIES with shadow-page-based recovery.

BIBLIOGRAPHIC NOTES

Our discussion of the ARIES recovery algorithm is based on [544]. [282] is a survey article that contains a very readable, short description of ARIES. [541, 545] also discuss ARIES. Fine, granularity locking increases concurrency but at the cost of l110l'e locking activity; [542] suggests a technique based on LSNs for alleviating this problrYl. [458] presents a forl11al verification of ARIES.

[355] is an excellent survey that provides a broader treatrnt of recovery aloritlulls than our coverage, in which we chose to concentrate on one particular algorithrn. [17] considers perforrnance of concurrency control and recovery algorithrlls, taking into account their interactions. The irnpact of recovery on concurrency control is also discussed in [769]. [625] contains a performance analysis of various recovery techniques. [236] cornpares recovery techniques for main rnerllory database systeulS, which are optirnized for the case that l110st of the active data set fits in rnain Hlemory.

[478] presents a description of a recovery algorithm based on write-ahead logging in which 'loser' transactions are first undone and then (only) transactions that corl1nlitted before the crash are redone. Shadow paging is described in [493, 337]. A scherne that uses a cOlmbination of shadow paging and in-place updating is described in [624].